

Lab 3 instructions — Build a Live Data Robot (OPIM 5512)

Module 3 · keep this open beside VS Code. Every click you need tonight, in order. You'll write a tiny **API** for one live data feed, merge it through the **GitHub workflow** you know (branch → commit → push → pull request → review → merge), and hand it to a **robot** that runs it every 5 minutes.

Your repo must be PUBLIC. Public repos get unlimited free robot minutes. A private repo running every 5 minutes would use up its free minutes in about a week.

Before we start — check three things

1. **GitHub Desktop opens and you're signed in.**
2. **VS Code opens** with the **Python** and **Jupyter** extensions (Extensions icon on the left → search each → Install).
3. You remember the Lab 1/2 loop. Same moves tonight; only the files change.

Pick roles: Partner A owns **energy** (ISO-NE). Partner B owns **weather** (National Weather Service).

Laptop Python fighting you? Don't lose the lab to setup. On your repo's page: **Code** → **Codespaces** → **Create codespace on main**. That's VS Code in the browser with Python and every package already installed. Every step below works the same there (use its **Source Control** panel instead of GitHub Desktop).

Part 1 — Set up the shared repo (Partner A drives, ~10 min)

1.1 Make the repo from the template

Open <https://github.com/drdave-teaching/opim5512-lab3-template> → green **Use this template** → **Create a new repository**.

Field	What to put
Owner	you (your personal account)
Repository name	opim5512-lab3-<netidA>-<netidB>
Visibility	Public (free robot minutes, and the notebook reads the data over a public link)

You now have the robot, the tests, the notebook, and the two API files with TODOs in them. **You built none of it — yet.**

1.2 Add your partner — then STOP until they accept

Repo **Settings** → **Collaborators** → **Add people** → Partner B's GitHub username → **Add**.

🛑 **STOP. Partner B: accept the invite RIGHT NOW** — check email (and spam), or the 🔔 on github.com → **Accept invitation**. Until you accept, you can't push, you won't appear as a reviewer, and the repo won't show up to clone.

1.3 Protect `main`

Repo **Settings** → **Rules** → **Rulesets** → **New ruleset** → **New branch ruleset**: - Name: `protect main` ·

Enforcement: **Active** - Target branches → **Add target** → **Include default branch** - Rules: ☒ **Require a pull request before merging** → **Required approvals: 1** → **Create**

(*Rehearsing solo with one account? Set Required approvals to 0.*) This only protects `main` — the robot writes to its own `data` branch, so it's never blocked.

1.4 Both partners: clone it — once

GitHub Desktop → **File** → **Clone repository** → **GitHub.com tab** → find the repo (🔄 refresh if needed) → **Clone**.

1.5 Each partner: make your branch — BEFORE you touch any file

Current branch → **New branch** → `dev-energy` (A) / `dev-weather` (B) → **Create branch** → **Publish branch** → **Fetch origin**.

🔴 Read the top bar before every commit. If it says `main`, stop and switch. **Already edited on `main` by accident? **Current branch** → **New branch** → `dev-...`, and when GitHub Desktop asks "Bring my changes to the new branch," say yes. Nothing lost.**

Part 2 — Build your API (each partner, on your own branch, ~30 min)

2.1 Open the repo in VS Code

GitHub Desktop → **Repository** → **Open in Visual Studio Code**. Then **Terminal** → **New Terminal** and install the packages once:

```
pip install -r requirements.txt
```

(Windows says `pip / python` not found? Use `py -m pip install -r requirements.txt` and `py` instead of `python` below.)

2.2 Partner A — `src/isone.py`, the energy API

ISO-NE's public report won't hand over its CSV unless you first **visit the report page** (to get a cookie) and then ask for the file **with that page as the Referer**. The file also mixes comment rows (`"C"`), header rows (`"H"`), and a trailer (`"T"`) in with the data rows (`"D"`). Your function hides all of that.

Fill in the three TODOs (the comments say exactly what goes there), then **delete the `raise`**

`NotImplementedError` line: - **TODO 1** — `session.get(...)` the report page (one line) - **TODO 2** — `resp =`

`session.get(url, headers={"Referer": REPORT_PAGE}, timeout=30)` (one line) - **TODO 3** — loop over `resp.text.splitlines()` and keep the lines that start with `"D"`

2.3 Partner B — `src/nws.py`, the weather API

The weather service asks every caller to identify itself (the `User-Agent` header) and answers in **nested JSON, in Celsius and km/h**. Your function hides that. Fill in the two TODOs, then **delete the** `raise NotImplementedError` line: - **TODO 1** — `resp = requests.get(url, headers=..., timeout=30)` (one line) - **TODO 2** — add the four readings to the row: `temp_f`, `dewpoint_f`, `humidity_pct`, `wind_mph`, using the helpers `c_to_f`, `round_or_none`, `kmh_to_mph`

Stamford: also change `STATION = "KBDL"` (Bradley) to `STATION = "KBDR"` (Sikorsky). The grid is shared; the weather is local.

2.4 Run it — see live data from right now

```
python src/isone.py      # Partner A
python src/nws.py        # Partner B
```

You should see the newest rows, stamped within the last few minutes (energy) or the last ~20 minutes (weather — it publishes late). Then run the same checks your pull request will run:

```
python -m pytest -v tests
```

Yours should **PASS**; your partner's will say **SKIPPED** until they merge. Skipped is not failed.

2.5 Commit and push

GitHub Desktop: top bar = **your dev- branch** → Summary `build the energy API` / `build the weather API` → **Commit** → **Push origin**.

Part 3 — Review & merge (both, ~15 min)

3.1 Open your pull request

github.com shows a yellow bar: **Compare & pull request** → check it's `dev-energy` → `main` (or `dev-weather`) → **Create pull request** → **Reviewers** → your partner.

★ **Don't skip Reviewers.** No reviewer = no approval = no two-way loop in the network graph — and the network graph is part of what you submit.

3.2 Watch the check

A few seconds after you open the PR, **tests** starts running at the bottom of the page. That's GitHub running `tests/` on your code in the cloud. ✓ **green** = your API returned the right columns, real rows, sensible units. ✗ **red** = click **Details** → find the red `FAILED` line; it says what's wrong in plain English.

3.3 Review your partner's

Open your **partner's** PR → **Files changed**. Read their function — could you explain what it hides? Check for the green ✓. **Review changes** → **Approve** → **Submit review**. Leave one real comment.

3.4 Merge, delete, pull

- **Merge pull request** → **Confirm** → **Delete branch**. Both PRs.
- **Both**: GitHub Desktop → **Current branch** → **main** → **Fetch origin** → **Pull origin**.

Part 4 — Wake the robot and watch it work (both, ~20 min)

4.1 Run the robot once by hand

github.com → your repo → **Actions** tab → **collect** (left side) → **Run workflow** → **Run workflow**. It also runs by itself about every 5 minutes (GitHub can take a while to start the schedule on a new repo, and runs can be late) — this just skips the wait.

4.2 Find its work on the `data` branch

When the run shows ✓: **Code** tab → branch menu (says `main`) → switch to `data` → open `isone_5min.csv` and `weather_obs.csv`. Click **commits** on the `data` branch — every one is by **lab3-bot**. Your `main` stayed clean.

The robot pulls the **whole recent window** every run and keeps only rows it hasn't seen (and refreshes rows the feed revised). So a late, doubled, or skipped run never breaks anything — the next one catches up. "No new rows this run" is correct, not broken.

4.3 Watch it live

In VS Code (on `main`), open `notebooks/Lab3_Watch_Your_Live_Data.ipynb` → **Select Kernel** (top right → your Python) → set `OWNER` and `REPO` → **Run All**. - **How fresh is it?** Energy is a few minutes old; weather ~20 minutes (it publishes late). - **The solar gap:** *Total Load* is what the grid delivered; *Total Load With Estimated Solar* adds back rooftop panels behind the meter. At midday the gap is thousands of megawatts. - **Come back in an hour and Run All again**. More rows. Tomorrow, a lot more.

4.4 Submit

Screenshot **Insights** → **Network** (the loop going both ways) **and** the `data` branch's commit list (the robot's work).

When it breaks

Symptom	Fix
<code>NotImplementedError</code>	You filled in the TODOs but didn't delete the <code>raise NotImplementedError</code> line.
'NoneType' object has no attribute 'raise_for_status'	TODO 1/2 still says <code>resp = None</code> — replace it with the real request.
<code>python</code> / <code>pip</code> not found (Windows)	Use <code>py</code> and <code>py -m pip</code> . Still stuck after 5 minutes? Switch to Codespaces .
<code>ZoneInfoNotFoundError</code> (Windows)	<code>pip install -r requirements.txt</code> (it installs <code>tzdata</code> , the time-zone database Windows lacks).
PR check X <code>temp_f=16.0</code> but the API says <code>15 C = 59.0 F</code>	You stored Celsius. Wrap it: <code>c_to_f(p["temperature"]["value"])</code> .
PR check X <code>no rows came back</code>	TODO 3 isn't keeping the <code>"D"</code> lines (mind the quotes: the line starts with <code>"D"</code> <i>including</i> the quote marks).
Your partner's tests say SKIPPED	Normal until they merge.
Push rejected on <code>main</code>	You're on <code>main</code> . Make/switch to your <code>dev-</code> branch — the rejection is the protection working.
Committed or dragged files while on <code>main</code>	New branch → "Bring my changes to the new branch" → yes.
Partner not in the Reviewers list	The invite was never accepted — back to 1.2 .
Robot run ✓ but "No new rows this run"	Correct — the feed hadn't published anything newer. Wait 5 minutes.
Notebook: HTTP Error 404	<code>OWNER / REPO</code> is wrong, or the robot hasn't run yet (no <code>data</code> branch). Do 4.1 first.
Notebook asks for a kernel / <code>ipykernel</code> missing	<code>pip install -r requirements.txt</code> , then Select Kernel again.

Never used this workflow? The Lab 1 kit walks every GitHub move slowly: [Lab1_FirstCommit](#).